

PBL Secondary Research Report

Systolic Array Matrix Multiplication

Chapter 1: Introduction

1.1 Background:

- A systolic array is used for matrix multiplication because it allows data to flow smoothly through a grid of small processing units, reducing the need to repeatedly access memory and speed up calculations.
- On FPGAs, this design fits perfectly since the hardware can be customized to run many operations in parallel, making the process faster and more energy-efficient.
- In simple terms, systolic arrays are chosen because they make big matrix calculations quicker, more organized, and less power-hungry compared to other methods.

1.2 Problem Statement:

Matrix-matrix multiplication is a fundamental operation in scientific computing, digital signal processing, and machine learning. However, traditional sequential algorithms for multiplying large matrices suffer from high computational complexity ($O(n^3)$) and memory bottlenecks, making them inefficient for real-time applications and large-scale data processing. As datasets grow and applications demand faster throughput, there is a pressing need for parallel hardware architectures that can perform matrix operations more efficiently.

1.3 Objectives:

- Reduce computation time: Traditional sequential matrix multiplication requires $O(n^3)$ operations. Systolic arrays exploit parallelism to bring this down to $O(n)$

- **Pipeline efficiency:** Data flows through processing elements (PEs) in a rhythmic, synchronized manner, ensuring continuous computation without idle cycles.
- **Energy Efficiency:** Reduce memory delays by passing data directly between processing units, improve energy efficiency compared to CPUs and GPUs, and provide flexibility to handle different matrix sizes and applications.

Chapter 2: Literature Review (Secondary Research)

2.1 Review of Existing Work:

1. Fault Tolerance in Large Arrays

- As systolic arrays scale, faulty processing elements (PEs) become a reliability issue.
- Traditional fault-tolerant mechanisms only worked with limited faulty PEs.
- Newer approaches use pair-matching mechanisms, where a fault-free PE acts as a proxy for a faulty one.
- This significantly improves reliability and reduces circuit area, making systolic arrays more practical for large-scale scientific and engineering applications.

2. FPGA Implementations

- FPGA-based systolic arrays leverage parallelism, pipelining, and resource sharing to accelerate matrix multiplication.
- These implementations are parameterizable, meaning they can adapt to different matrix sizes.

2.2 Comparative Analysis:

Paper	Architecture used	Throughput (MSPS)	Area	Limitations
VLSI-based and RTL-based systolic array implementations	Grid architecture of Processing Elements (PEs),	<u>VLSI-Based</u> - 500 MSPS – several GSPS <u>RTL-Based</u> - 40–250 MSPS	Low (compact silicon footprint) High (large LUT/DSP utilization)	Design complexity vs limited hardware optimization
Design and Implementation of 4×4 Systolic Array using Verilog	MAC-based systolic array with pipelined data flow	~2.5 Gb/s at 100 MHz (approximate)	FPGA resource-based (LUTs, FFs, DSPs)	Limited hardware optimization, pipeline latency
VLSI Design of 4×4 Systolic Array for Matrix Multiplication	DPU-based systolic array with custom multiplier, adder, and register units at circuit level	High throughput due to parallel processing and optimized data flow	~22,254 μm^2 (layout-based silicon area)	High design complexity, layout challenges, and signal integrity issues
Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication	2D systolic array with MAC PEs	210.2 MHz ($\approx 2\times$ conventional)	Fewer multipliers, adders, registers	Fixed matrix size, scaling adds complexity

2.3 Research Gap Identification:

1. Scalability vs. Reliability

- Current systolic arrays scale well in size (e.g., 128×128 PEs in ASICs), but fault tolerance becomes a major challenge.
- **Gap:** Need for scalable, low-overhead fault tolerance methods for very large systolic arrays.

2. Energy Efficiency vs. Flexibility

- ASIC/VLSI designs are highly energy-efficient but fixed-function.
- FPGA/RTL designs are flexible but consume more area and power.
- **Gap:** Lack of architectures that balance energy efficiency with reconfigurability, especially for diverse workloads in AI and HPC.

Chapter 3: Proposed System Design

3.1 Design Justification:

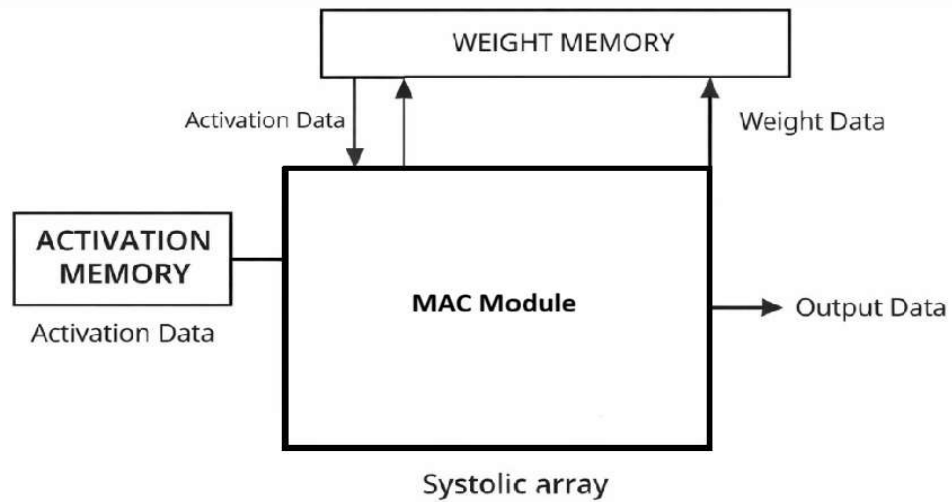
1. Parallelism and High Throughput

- Matrix multiplication involves repetitive multiply-accumulate (MAC) operations. A systolic array enables massive parallelism by assigning each MAC operation to a dedicated Processing Element (PE).
- Data flows rhythmically through the array, allowing multiple computations to occur simultaneously

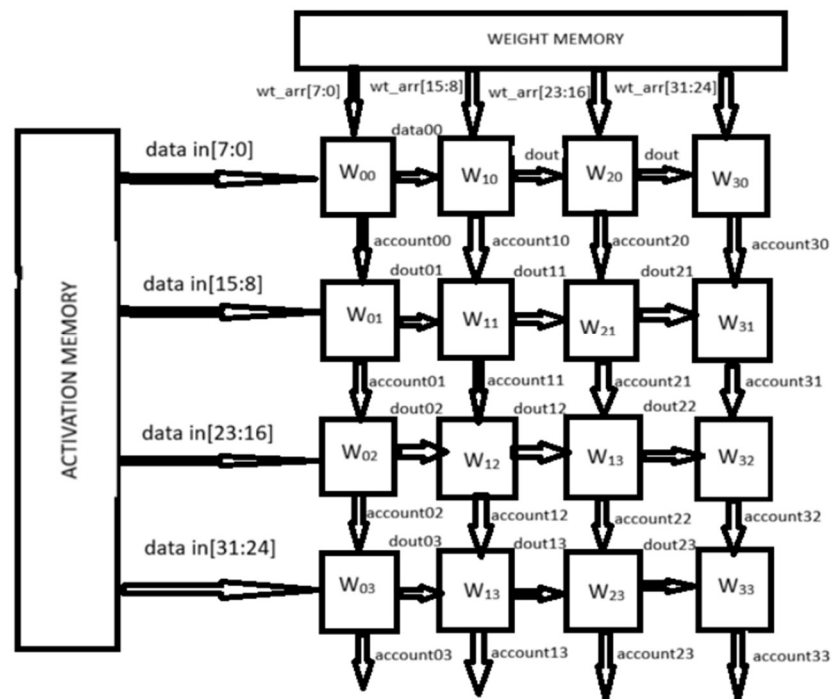
2. Local Data Movement

- Traditional architectures suffer from memory bottlenecks due to frequent global data transfers.
- In a systolic array, data moves locally between neighbouring PEs, minimizing global memory access.
- This localized communication reduces latency and power consumption, making the design ideal for real-time and embedded systems.

3.2 Block Diagram

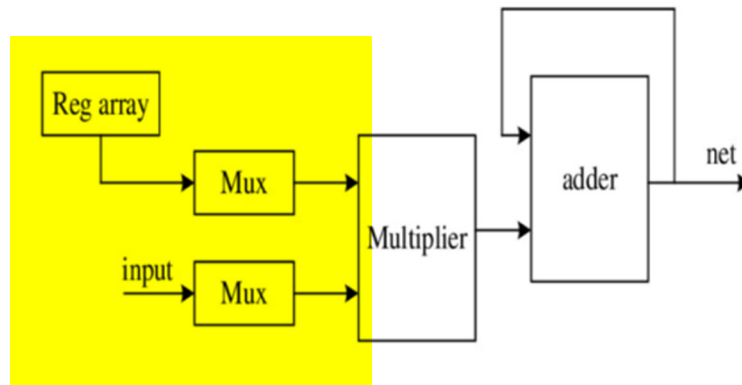


3.3 Architecture Diagram:



Chapter 4: Leaf Cell Design:

4.1 Identification of Leaf Cells:



MAC (Multiply-Accumulate) Module;

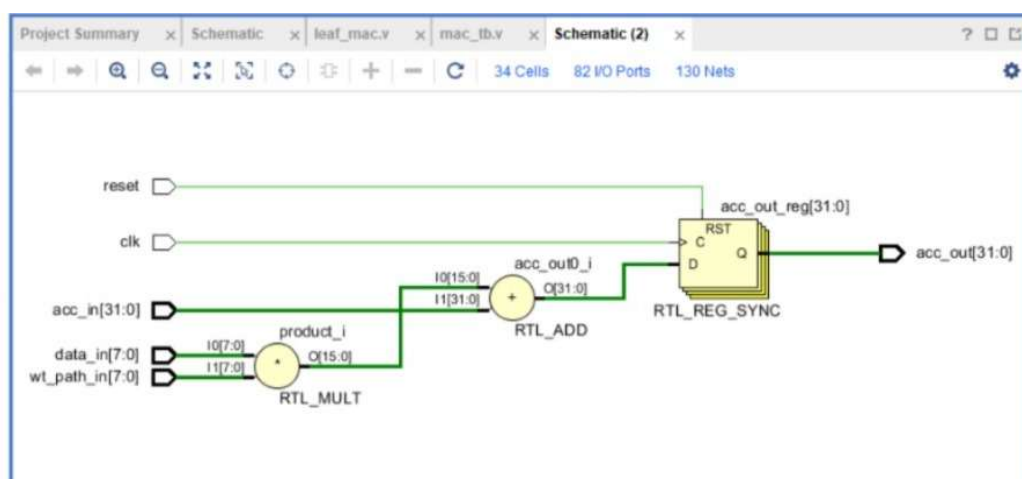
Leaf Cells:

- 2:1 Mux
- Multiplier
- Adder

4.2 Leaf Cell Description:

1) MAC Module

A MAC module, or Multiply–Accumulate module, is a small hardware unit designed to perform two operations in sequence: multiplication and addition. It takes two input values, usually an activation and a weight, multiplies them together, and then adds the result to an ongoing sum stored in an accumulator.



$$\text{Sum}_{i,j}(t+1) = \text{Sum}_{i,j}(t) + (A_{i,k} \times B_{k,j})$$

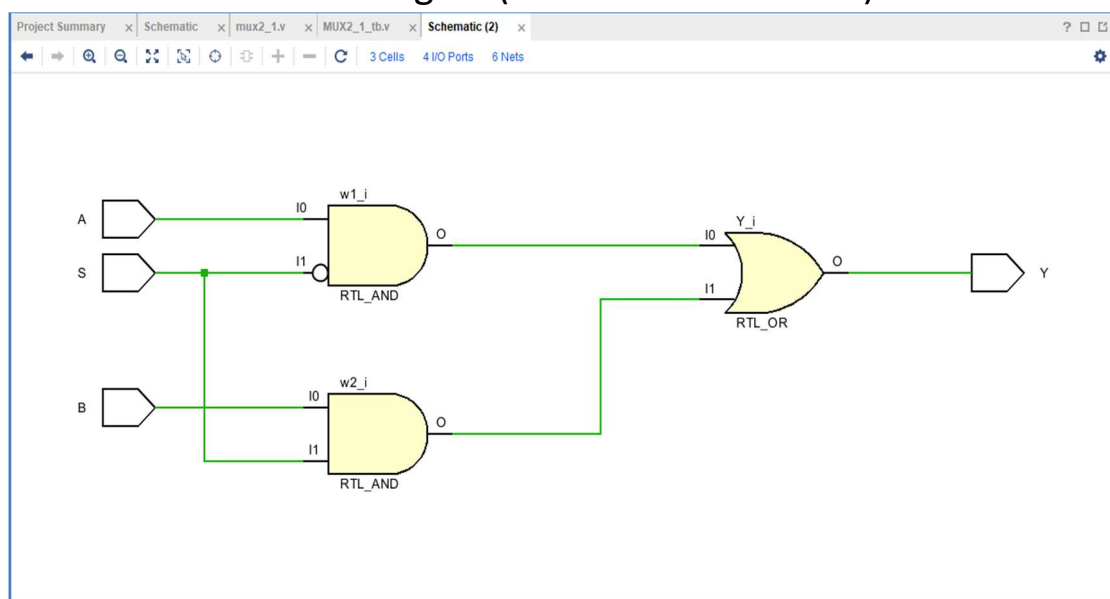
Multiply: It takes two inputs — usually an activation (data value) and a weight (coefficient) — and multiplies them.

Accumulate: The product is then added to a running sum (an accumulator register). This allows partial results to build up over time.

Output: The accumulated result is passed forward, either to memory or to the next stage in the array.

2) 2:1 Mux

2:1 multiplexer (mux) is a digital circuit that selects one of two input signals and forwards it to a single output line, based on the value of a control signal (called the select line).

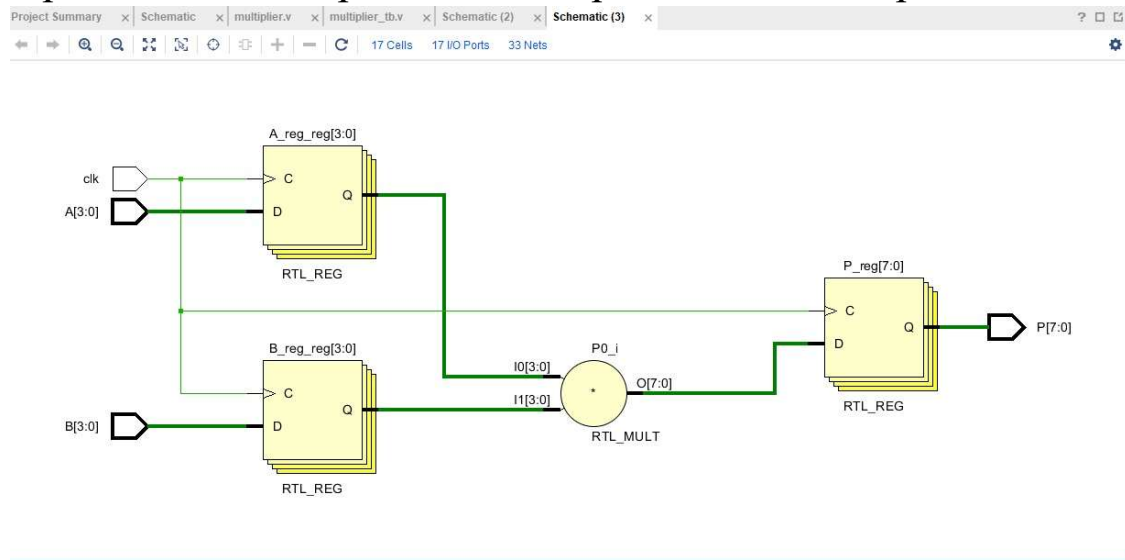


The mux has two inputs (A and B), one select line (S), and one output (Y). The select line decides which input gets passed to the output. The circuit uses two AND gates and one OR gate to implement. One AND gate takes A and the inverted select (S'). This means A will only pass through when S = 0. The other AND gate takes B and the select signal (S). This means B will only pass through when S = 1. The OR gate

combines the outputs of the two AND gates, so the final output Y is whichever input was enabled

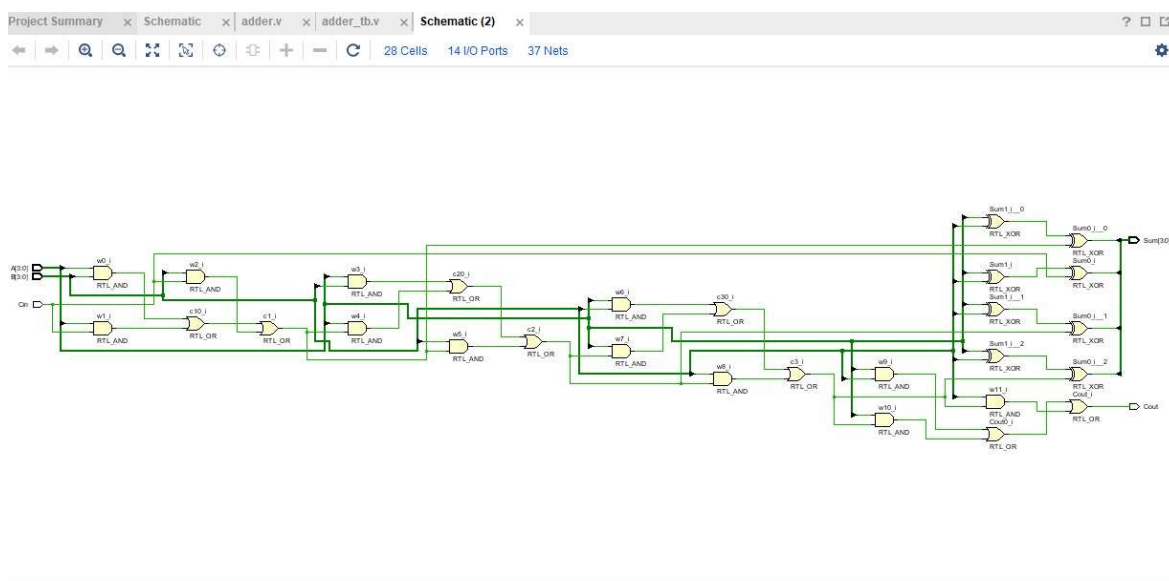
3) Multiplier

A multiplier is a digital circuit or arithmetic unit that takes two input numbers and produces their product as the output.



4) Adder

An adder is a basic digital circuit that performs the arithmetic operation of addition. It takes two binary numbers as inputs and produces their sum as the output, often along with a carry signal if the result exceeds the bit capacity.



4.3 Leaf Cell Simulation:

[illegible]

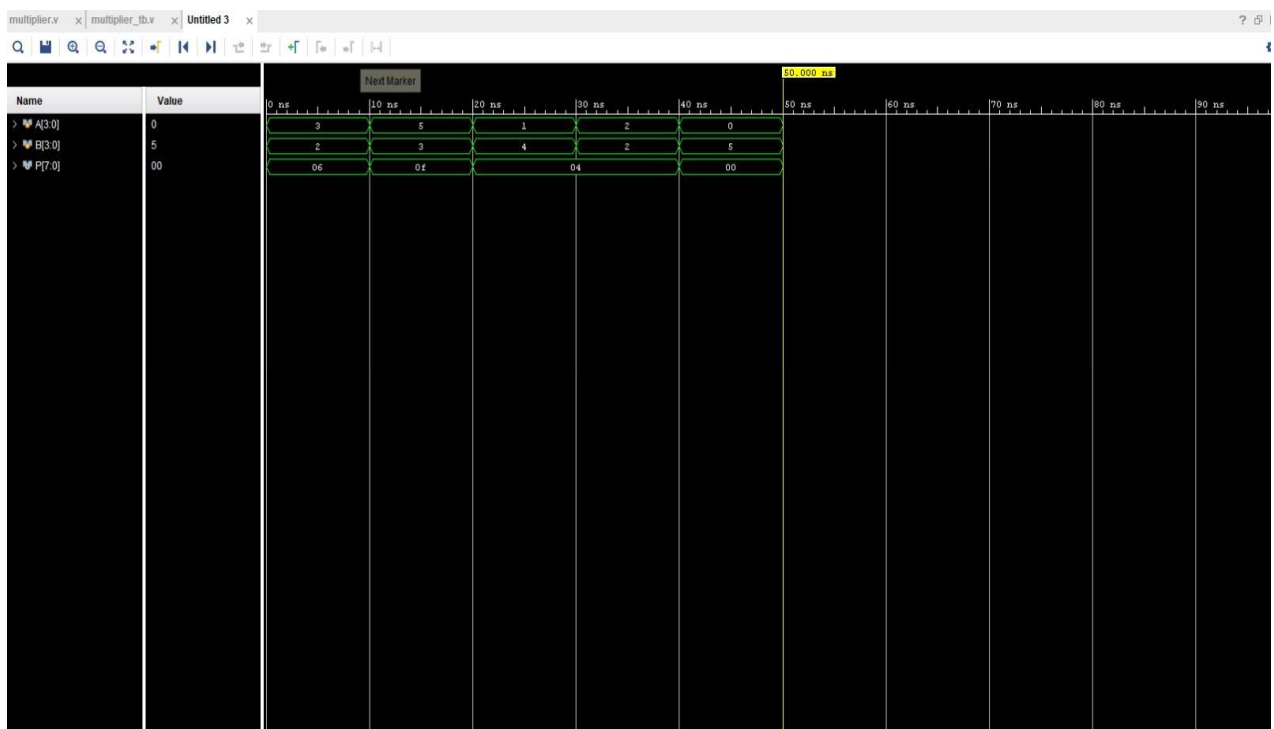
Case2 :data in:4, wt path: 5, acc in:10

Case:3 data in:10 , wt path: 10 , acc in 100

The screenshot shows the Intel Quartus II Timing Analyzer interface. The top toolbar includes icons for file operations, search, and simulation. The main window displays a timing diagram for a 100MHz clock. The diagram shows four signals: A, B, C, and Y. Signal A is a square wave, B is a square wave, C is a square wave, and Y is a square wave. The timing diagram shows that the signals are synchronized with the clock. The time range is from 0 ns to 70 ns, with major ticks every 10 ns. The signals are plotted against a background grid.

A	B	S	Y		A	B	S	Y
0	0	0	0		0	0	1	0
0	1	0	0		0	1	1	1
1	0	0	1		1	0	1	0
1	1	0	1		1	1	1	1

3) Multiplier



A = 4'd3; B = 4'd2;

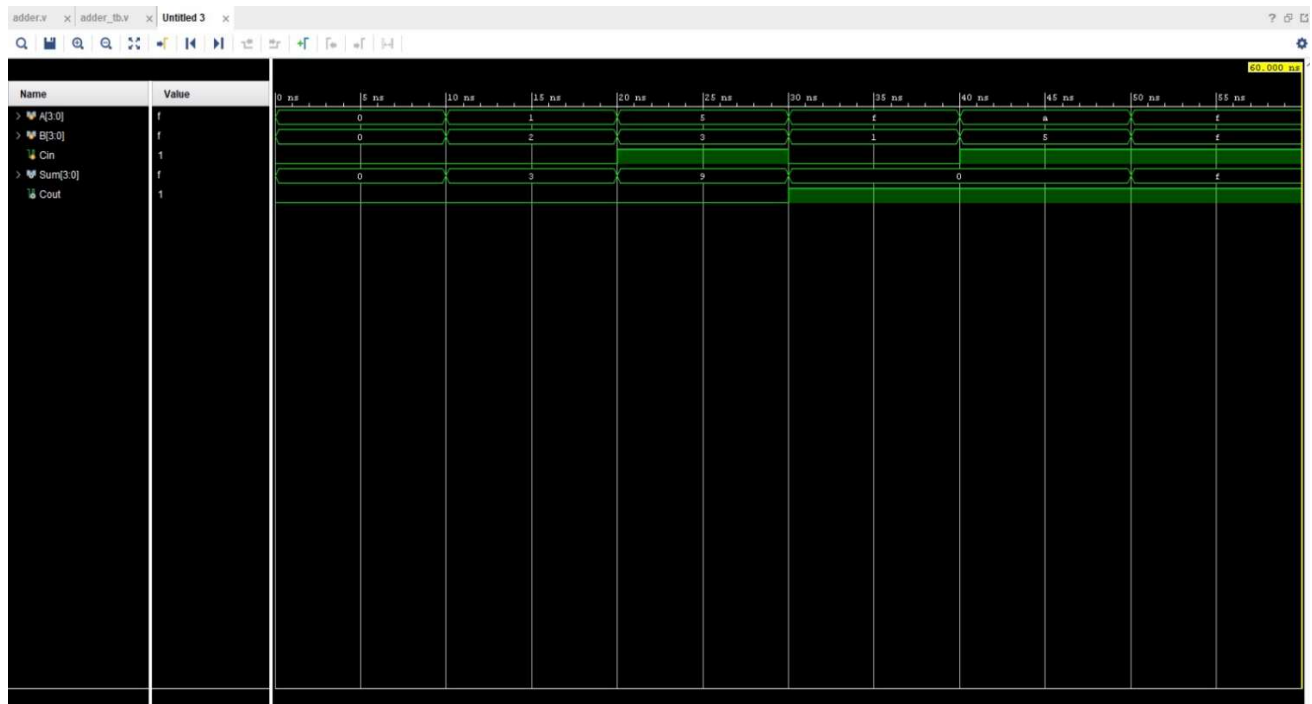
A = 4'd5; B = 4'd3;

A = 4'd1; B = 4'd4;

A = 4'd2; B = 4'd2;

A = 4'd0; B = 4'd5;

4) Adder



```
A = 4'b0000; B = 4'b0000; Cin = 0;  
A = 4'b0001; B = 4'b0010; Cin = 0;  
A = 4'b0101; B = 4'b0011; Cin = 1;  
A = 4'b1111; B = 4'b0001; Cin = 0;  
A = 4'b1010; B = 4'b0101; Cin = 1;  
A = 4'b1111; B = 4'b1111; Cin = 1;
```

Appendix

- Tool Used – Vivado
- References: -
 - Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication: Mahendra Vucha ,Research Scholar, MANIT, Bhopal and Asst. Prof, Dept. EC, GGITM, Bhopal, India.

- ADiP: Adaptive-Precision Systolic Array for Matrix Multiplication Acceleration, Ahmed J. Abdel Maksoud, Member, IEEE
- DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration: Ahmed J. Abdelmaksoud, Shady Agwa, Themis Prodromakis
- Construction of a 4x4 Systolic Array Kyle E. Berger, Abhishek Damle, and Jack Retcher